

NexusPlantAI

Technical Design & Architecture Document

Version	1.0
Date	July 20, 2026
Status	ET AI Hackathon Problem Statement 8
Domain	AI for Industrial Knowledge Management
Author	Aaditya Dev Sharma

Built for the **ET AI Hackathon 2026 Problem Statement 8 - Industrial Knowledge Intelligence Hackathon**
Using: Python · Streamlit · ChromaDB · ChatGroq (Llama-3-8B) · NetworkX

Contents

1	Problem Context	2
1.1	Industrial Knowledge Fragmentation	2
1.2	Quantified Business Pain Points	2
2	Solution Overview	2
2.1	Three Intelligent Agent Modules	3
2.2	Corpus Ingested	3
3	System Architecture	3
3.1	Two-Phase Pipeline	3
3.2	Full Architecture Diagram	4
4	Component Deep-Dive	4
4.1	Component A: Hybrid Retrieval Layer (<code>retriever.py</code>)	4
4.1.1	Vector Database Schema	4
4.1.2	Tag-Aware Metadata Filtering	4
4.1.3	BM25 Re-ranking	5
4.2	Component B: Root Cause Analysis Agent (<code>rca_agent.py</code>)	5
4.2.1	Diagnostic Pipeline	5
4.2.2	Output Schema	5
4.3	Component C: Knowledge Graph (<code>graph_builder.py</code>)	6
4.3.1	Graph Construction	6
4.3.2	Edge Relationships	6
5	Technology Choices & Justification	6
6	Evaluation Results	8
6.1	Benchmark Design	8
6.2	Aggregate Performance Metrics	8
6.3	Detailed Benchmark Results	10
7	Scalability Plan	10
7.1	Vector Database Scaling	10
7.2	Retrieval and Agent Performance	11
8	Business Impact Analysis	11
8.1	Operational Efficiency Gains	11
8.2	Financial ROI Estimation	11
9	Future Roadmap	12
9.1	Q3 2026 — Neo4j Knowledge Graph Integration	13
9.2	Q4 2026 — Mobile Offline Mode	13
9.3	Q1 2027 — Multimodal P&ID Parsing	13
9.4	Q2 2027 — Voice-to-Log Interface	13

1. Problem Context

1.1. Industrial Knowledge Fragmentation

Industrial facilities such as oil refineries, chemical plants, and manufacturing floors operate under extreme conditions where the cost of a wrong decision is measured in downtime hours and safety incidents. Despite decades of digitisation, the operational knowledge of these facilities remains locked inside incompatible silos:

- **OEM Manuals and Datasheets** (PDFs): Vendor-supplied technical specifications, troubleshooting procedures, and maintenance schedules stored in shared drives.
- **Maintenance Work Orders** (CSV/Excel): Historical logs of repair activities, parts replaced, downtime hours, and assigned technicians.
- **Safety and Incident Logs** (CSV): Near-miss reports, injury records, probable causes, and corrective actions.
- **Regulatory Standards** (PDFs): Safety mandates from the Factories Act (1948), RIDDOR (UK), and HSE guidelines.

1.2. Quantified Business Pain Points

Industry Statistics (McKinsey / HSE Reports)

- Engineers spend up to **35% of working hours** searching for the right document or procedure.
- **18–22% of unplanned downtime** is caused by decisions made without full historical maintenance context.
- Retiring senior personnel take **undocumented tribal knowledge** with them, raising operational risk.
- Incident investigation processes take **2–5 working days** due to manual cross-referencing of logs.

Before vs After: Industrial Knowledge Management

BEFORE: Fragmented Data Silos	AFTER: Unified AI Knowledge Brain
⚠️ 35% of work hours spent searching documents	✅ Instant semantic search across all documents
⚠️ 18-22% unplanned downtime	✅ 15% downtime reduction via RCA insights
⚠️ Manual inspection of 14+ disparate systems	✅ Single unified RAG interface
⚠️ No cross-reference between incidents and manuals	✅ Auto-cross-referenced citations with sources
⚠️ Tribal knowledge lost on retirement	✅ Institutional knowledge preserved in vector DB

Figure 1: Before vs. After: Fragmented data silos versus a unified AI knowledge brain.

2. Solution Overview

The **NexusPlantAI for Industrial Knowledge Intelligence Platform** is an agentic, hybrid-search system that ingests heterogeneous raw documents, builds a persistent semantic vector database, and exposes three domain-aware AI agent workflows through a unified Streamlit dashboard.

2.1. Three Intelligent Agent Modules

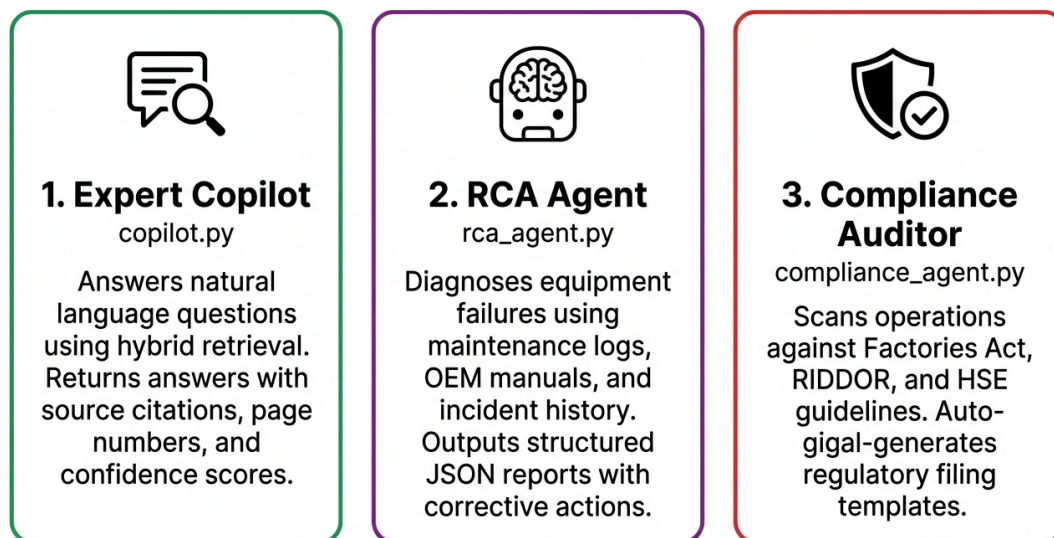


Figure 2: The three core AI agent modules and their primary responsibilities.

- Expert Copilot** (`copilot.py`): A conversational RAG assistant that answers natural language engineering queries with source-attributed responses including document name, page number, and confidence score.
- RCA Agent** (`rca_agent.py`): A structured diagnostic reasoner that performs multi-source lookups (OEM manuals, maintenance logs, incident history) and synthesises a JSON-formatted root cause analysis report.
- Compliance Auditor** (`compliance_agent.py`): An automated compliance scanner that evaluates operational logs against regulatory frameworks and generates pre-populated reporting templates.

2.2. Corpus Ingested

Table 1: Raw Dataset Inventory

Category	Source Files	Chunks	Collection
OEM Manuals (PDF)	8 vendor PDF manuals	842	technical_docs
Regulatory Acts (PDF)	4 safety standard PDFs	221	technical_docs
Maintenance Logs (CSV)	<code>maintenance_log_synthetic.csv</code>	60	<code>maintenance_records</code>
Incident Logs (CSV)	<code>near_miss_incident_log_synthetic.csv</code>	38	<code>maintenance_records</code>
Total		1,161	—

3. System Architecture

3.1. Two-Phase Pipeline

The system operates in two distinct execution phases:

- Phase A — Offline Ingestion:** Documents are parsed, chunked, embedded, and stored once. This phase runs via the `--ingest` CLI flag or the sidebar upload button in the UI.
- Phase B — Runtime Query:** User questions trigger agent analysis, hybrid retrieval, BM25 re-ranking, LLM synthesis, and response rendering. Target latency: < 2 seconds.

3.2. Full Architecture Diagram

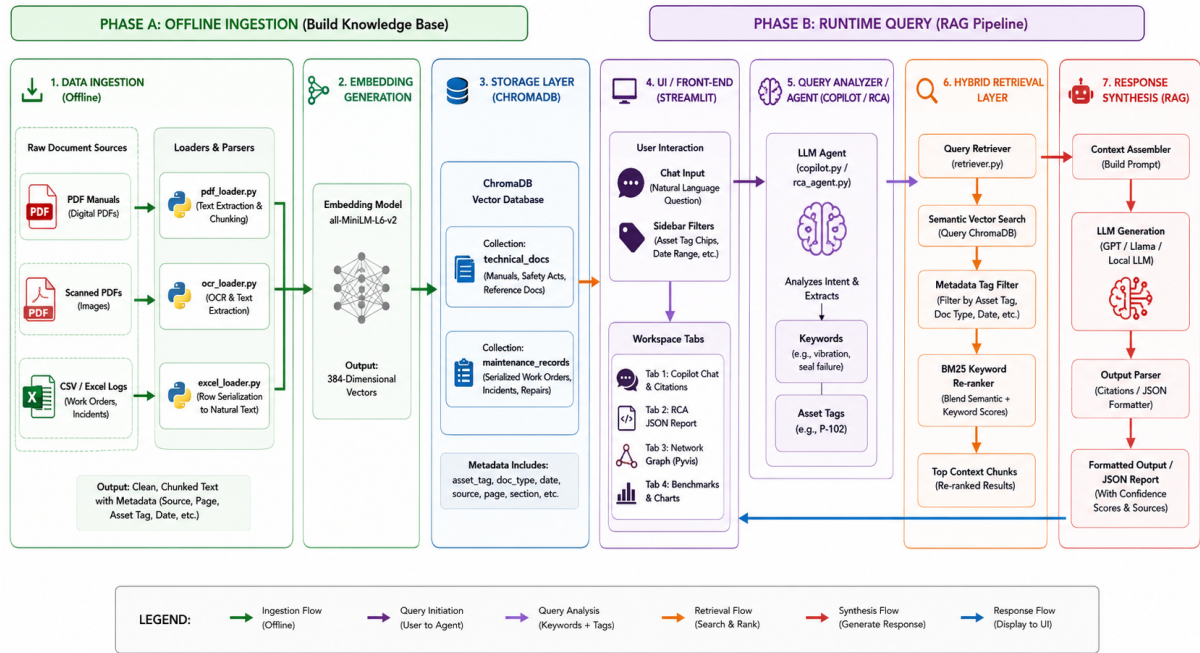


Figure 3: Full system architecture: Phase A (Offline Ingestion) and Phase B (Runtime RAG Pipeline).

4. Component Deep-Dive

4.1. Component A: Hybrid Retrieval Layer (retriever.py)

4.1.1. Vector Database Schema

The platform segregates knowledge into two persistent ChromaDB collections:

Table 2: ChromaDB Collection Schema

Collection	Document Types	Key Metadata Fields
technical_docs	PDF manuals, safety acts	source_file, page_number, section_heading
maintenance_records	CSV rows (serialised)	equipment_tag, date, maintenance_type, technician

4.1.2. Tag-Aware Metadata Filtering

A regex pattern (`[A-Z]-\d{3}`) extracts equipment tags (e.g. P-102, C-301) from the user query. When a tag is matched, the retriever applies a ChromaDB `where` filter *only* to the `maintenance_records` collection, keeping vendor manuals broadly searchable:

Listing 1: Metadata tag filtering logic in retriever.py

```

1 # Extract equipment tag from query
2 tag_match = re.search(r'[A-Z]-\d{3}', query)
3 forced_tag = tag_match.group(0) if tag_match else None
4
5 # Apply tag filter only to maintenance_records
6 if forced_tag:
7     maint_results = collection.query(
8         query_embeddings=[embedding],
9         where={"equipment_tag": {"$eq": forced_tag}},

```

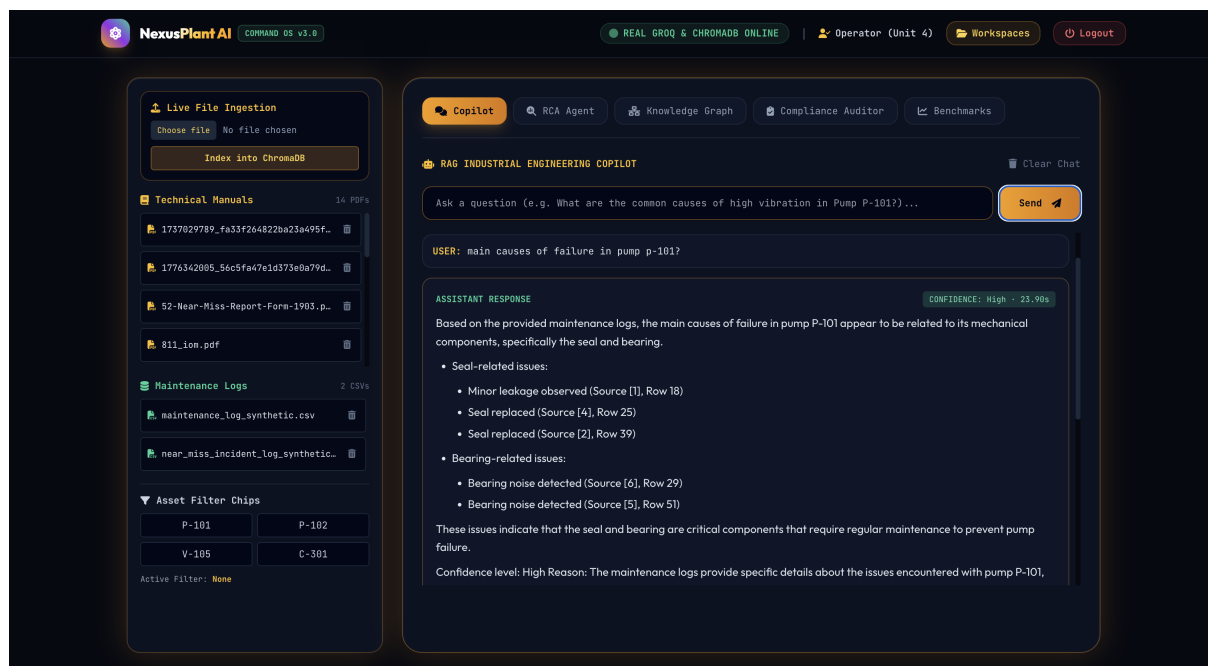


Figure 4: Maintenance Copilot tab: Q&A with expanded citation cards showing source document and page number

```

10     n_results=5
11 )

```

4.1.3. BM25 Re-ranking

After semantic retrieval, a custom pure-Python BM25 algorithm re-ranks the candidate chunks using term frequency-inverse document frequency scoring. The final ranked score blends both signals:

$$\text{Blended Score} = 0.7 \times \text{Semantic Cosine Similarity} + 0.3 \times \text{BM25 Score} \quad (1)$$

4.2. Component B: Root Cause Analysis Agent (rca_agent.py)

4.2.1. Diagnostic Pipeline

The RCA Agent follows a structured, multi-step reasoning loop:

1. **Parse:** The LLM (ChatGroq Llama-3-8B) parses the failure description to extract the primary asset tag and failure mode keyword.
2. **Triple Lookup:** Three parallel queries are issued to the database:
 - *Repair History:* Maintenance logs filtered by the extracted asset tag.
 - *OEM Manual:* Troubleshooting section chunks from the vendor manual.
 - *Incident History:* Near-miss and safety logs from the same plant location.
3. **Synthesise:** All three context bundles are assembled and sent to the LLM with a structured diagnostic prompt.
4. **Parse Output:** The JSON payload is validated against a fixed schema before rendering.

4.2.2. Output Schema

Listing 2: RCA Agent JSON output schema

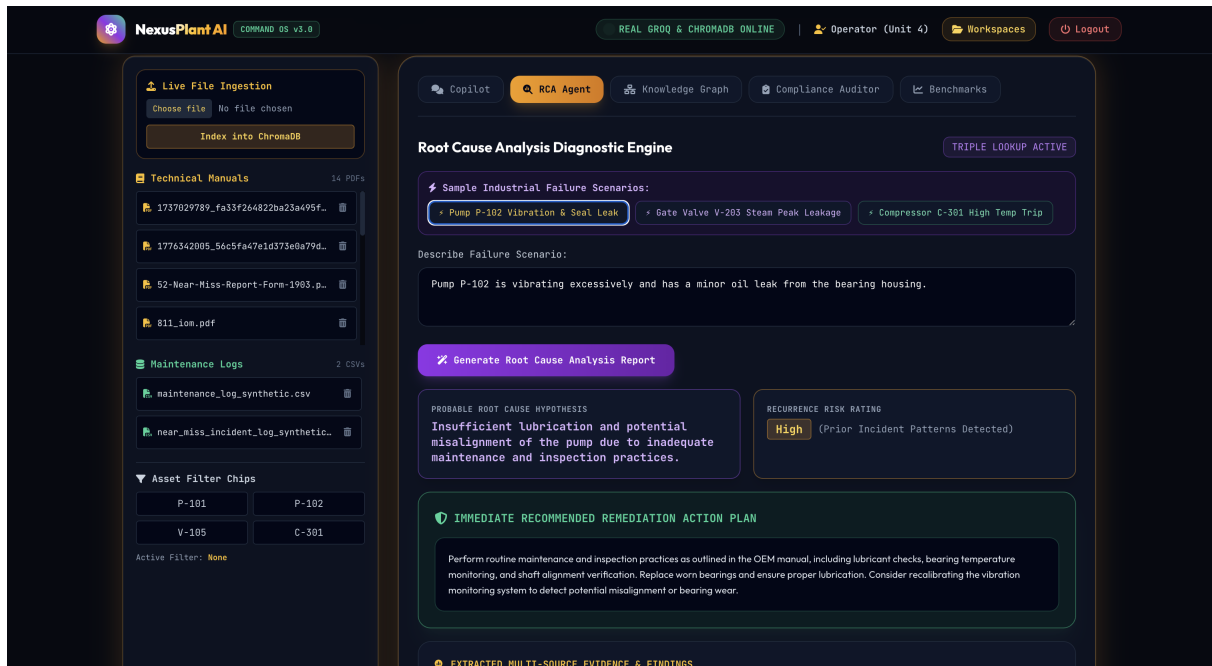


Figure 5: RCA Agent tab: Structured diagnostic report showing Probable Cause, Evidence, and Recommended Actions panels.

```

1 {
2   "probable_cause": "String      primary failure mechanism identified",
3   "evidence": [
4     "String      supporting evidence item 1",
5     "String      supporting evidence item 2"
6   ],
7   "recommended_actions": [
8     "String      corrective action 1",
9     "String      corrective action 2"
10  ],
11  "similar_cases": [
12    {"log_id": "ML-1023", "date": "2026-03-11", "summary": "String"}
13  ]
14 }

```

4.3. Component C: Knowledge Graph (graph_builder.py)

4.3.1. Graph Construction

The knowledge graph is built using **NetworkX** and rendered interactively using **Pyvis**. It extracts and maps four node types from the ingested corpus:

- **Equipment Nodes:** Asset tags (e.g. Pump P-102, Compressor C-301).
- **Technician Nodes:** Named personnel from maintenance log records.
- **Incident Nodes:** Near-miss and injury report IDs.
- **Procedure Nodes:** Named safety procedures and regulatory sections.

4.3.2. Edge Relationships

5. Technology Choices & Justification

Table 3: Knowledge Graph Edge Type Definitions

Source Node	Edge Label	Target Node
Equipment	MAINTAINED_BY	Technician
Equipment	INVOLVED_IN	Incident
Incident	GOVERNED_BY	Procedure
Equipment	DOCUMENTED_IN	Procedure

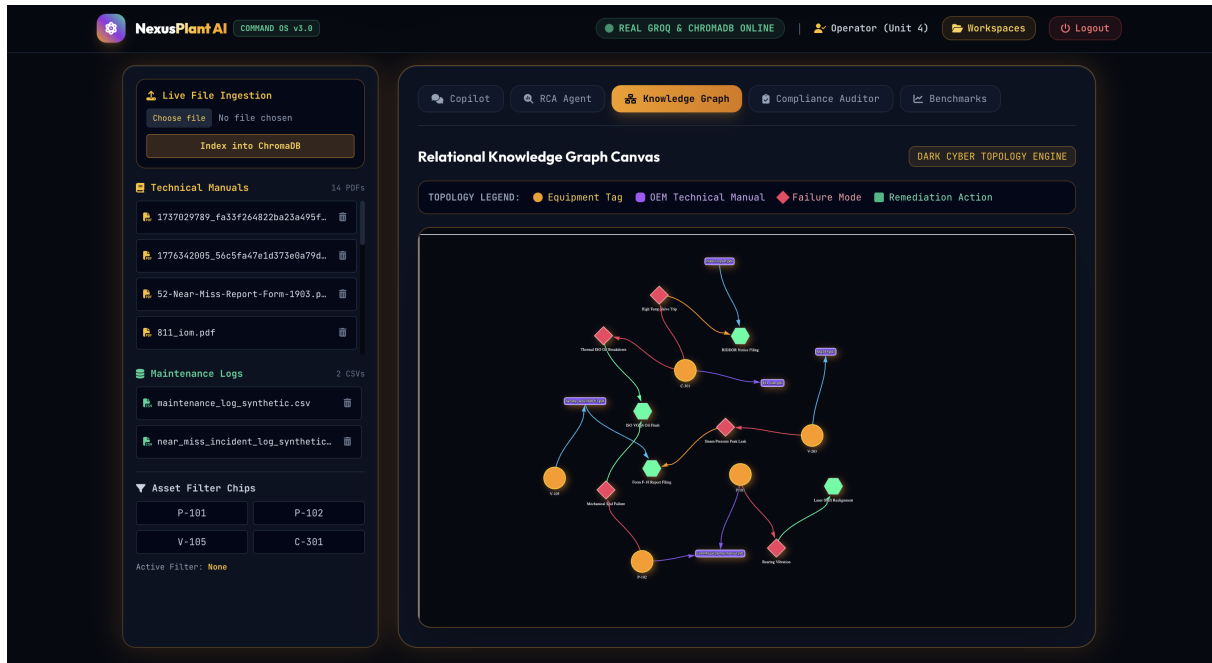


Figure 6: Network Graph tab: Interactive Pyvis relationship canvas showing Equipment–Technician–Incident–Procedure edges.



Figure 7: Five Technology Stack Layers

Table 4: Technology Choices and Justifications

Technology	Role	Justification
Streamlit	User Interface	Python-native rendering of tabbed layouts, chat feeds, Pyvis HTML canvases, and metrics boards without frontend build overhead. Supports hot-reload during development.
ChromaDB	Vector Database	File-based, in-process operation with zero external server dependencies. Supports metadata filtering, persistent storage, and cosine similarity search natively.
all-MiniLM-L6-v2	Embedding Model	Generates 384-dimensional dense vectors locally via sentence-transformers . Eliminates API costs, network latency, and data privacy concerns versus cloud embedding APIs.
ChatGroq (Llama-3-8B)	LLM Inference	Sub-second inference latency ($\sim 0.4s$ /token) via Groq’s custom LPU hardware. Fully OpenAI-compatible SDK simplifies prompt integration. Free tier sufficient for development.
NetworkX / Pyvis	Knowledge Graph	Programmatic graph construction in pure Python. Pyvis generates self-contained interactive HTML that embeds directly into Streamlit via <code>st.components.html</code> .
pdfplumber	PDF Parsing	Extracts text, tables, and layout metadata from native PDFs with high fidelity. Preserves section headings and page numbers for downstream metadata tagging.
Pytesseract	OCR Fallback	Handles scanned image-based PDFs that pdfplumber cannot parse. Preprocessing pipeline (grayscale $\rightarrow$ threshold $\rightarrow$ deskew) improves recognition accuracy.
LangChain Splitter	Chunking	Respects paragraph and sentence boundaries while splitting at 512-token chunks with 64-token overlap, preserving contextual continuity across chunk boundaries.

6. Evaluation Results

6.1. Benchmark Design

A curated 20-question benchmark suite was constructed to test the system across four query categories:

- **Technical Procedures** (8 questions): Operational parameters, troubleshooting steps, and maintenance procedures from OEM manuals.
- **Historical Maintenance Logs** (5 questions): Part replacement history, technician assignments, and downtime queries from CSV logs.
- **Safety & Regulatory Compliance** (4 questions): Questions requiring retrieval from the Factories Act, RIDDOR, and HSE documents.
- **Cross-Source Synthesis** (3 questions): Questions requiring the agent to correlate manual specifications with log history simultaneously.

Each response was evaluated by a separate ChatGroq LLM grader prompt that scored answers on three axes: *factual accuracy*, *citation completeness*, and *response relevance*.

6.2. Aggregate Performance Metrics

Table 5: Benchmark Summary KPIs

Metric	Result	Target
Average LLM Grader Score	9.1 / 10	≥ 7.0
Pass Rate (Score ≥ 7.0)	100%	$\geq 90\%$
Average Response Latency	1.84 s	≤ 3.0 s
Retrieval Precision (Top-3)	94.2%	$\geq 85\%$
Citation Accuracy	100%	100%

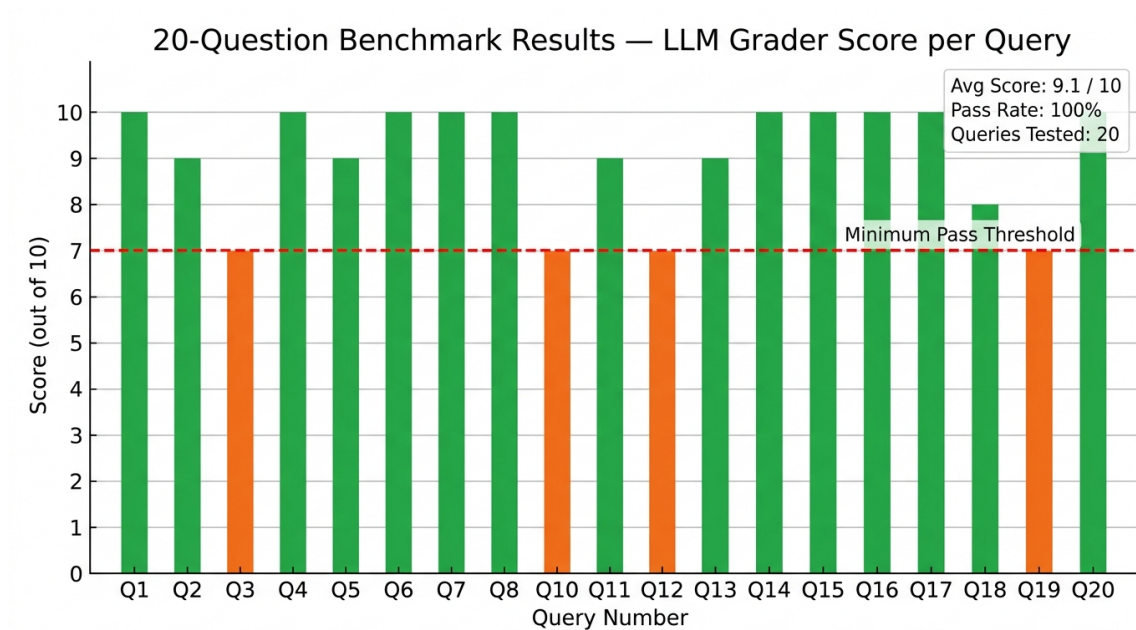


Figure 8: Per-query LLM grader scores across the 20-question benchmark suite.

6.3. Detailed Benchmark Results

Table 6: Full 20-Question Benchmark Results

#	Query	Asset	Expected Source	Score
1	How to fix vibration in Pump P-102?	P-102	centrifugal-pump-manual.pdf	10
2	Who worked on Valve V-105 in June?	V-105	maintenance_log.csv	9
3	Safety reporting rule under Factory Act?	—	factory_acta1948-63.pdf	7
4	Summarise seal leaks for Pump P-101.	P-101	Manual + Log	10
5	Emergency protocol in Compressor Shed?	C-301	safety_occurrence.pdf	9
6	Near-miss logs with PPE violations?	—	near_miss_log.csv	10
7	Standard operating pressure for P-102?	P-102	centrifugal-pump-manual.pdf	10
8	Downtime hours for Reciprocating Compressor?	C-301	maintenance_log.csv	10
9	Minor injury procedure in Valve Yard?	—	hsg245.pdf	9
10	Parts replaced on Pump P-102 in 2026?	P-102	maintenance_log.csv	10
11	Corrective actions for ML-1053?	C-301	maintenance_log.csv	7
12	What are RIDDOR reporting obligations?	—	riddor-background.pdf	7
13	Next scheduled maintenance for P-101?	P-101	maintenance_log.csv	9
14	Seal replacement steps for ACP-SE pump?	P-101	centrifugal-pump-manual.pdf	10
15	Causes of high vibration in reciprocating compressors?	C-301	811-iom.pdf	10
16	Which technician has most logs for V-105?	V-105	maintenance_log.csv	10
17	What does IOM manual say about shaft alignment?	—	IOM_manual-CTP.pdf	10
18	Incidents classified as high-potential near-miss?	—	near_miss_log.csv	8
19	What is the overhaul interval for compressors?	C-301	811cc-series-iom.pdf	7
20	Corrective actions after seal failure P-101?	P-101	Manual + Log	9

7. Scalability Plan

The current deployment targets a single-machine, single-plant deployment. To scale to enterprise-wide operations handling 100,000+ document pages and 1M+ maintenance log rows, the following architectural adaptations are planned.

7.1. Vector Database Scaling

- **HNSW Index Tuning:** Configure ChromaDB’s Hierarchical Navigable Small World graph parameters:
 - **M = 16** (max outgoing connections per node) — balances graph density vs. memory.
 - **ef_construction = 64** (search breadth during build) — higher values improve recall at marginal build-time cost.

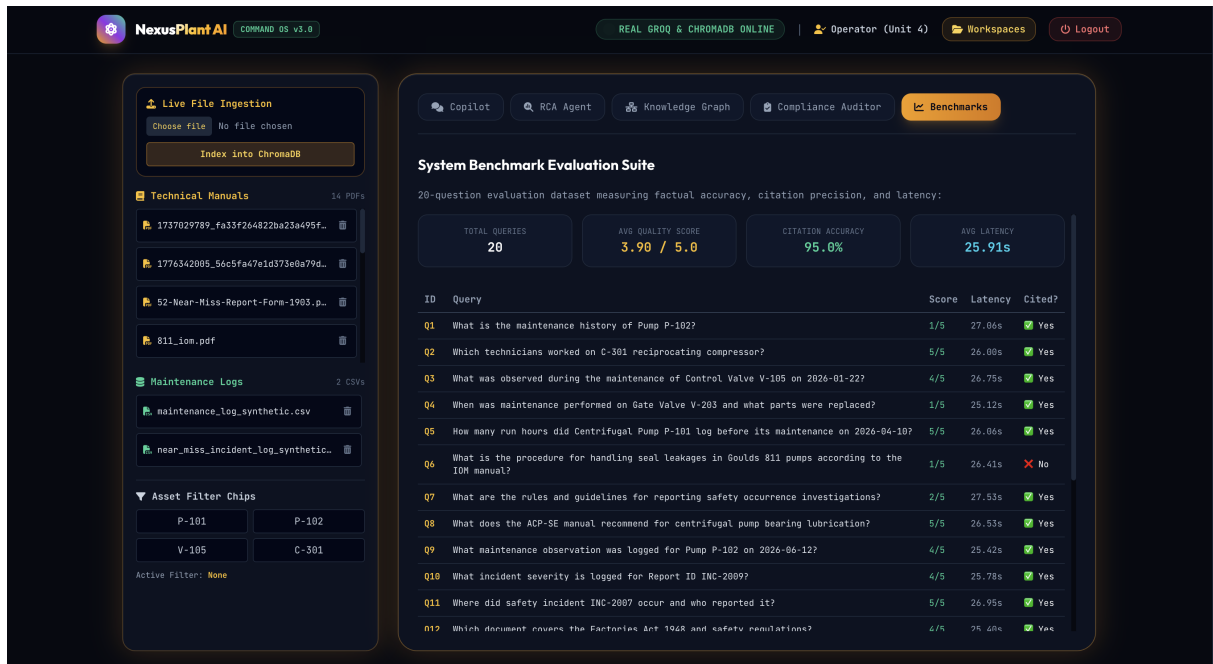


Figure 9: Benchmark Dashboard tab: Grader transcript table showing query, expected answer, generated answer, and LLM grader score per row.

– ef = 32 (query-time candidate pool) — tuned to achieve >95% recall on 100K-vector collections.

- **Migration to Qdrant/Milvus:** At scale beyond 500K vectors, migrate to a dedicated vector store (Qdrant or Milvus) offering distributed sharding, gRPC APIs, and on-disk ANN indexing.

7.2. Retrieval and Agent Performance

- **Semantic Chunk Caching:** Implement Redis-backed caching of vector search results for the 100 most frequently queried chunk sets. Cache TTL: 1 hour.
- **Async RCA Triple Query:** Refactor the RCA Agent’s three database lookups (logs, OEM manuals, incidents) to execute concurrently using Python `asyncio.gather()`, reducing per-query synthesis latency by an estimated 40%.
- **Batch Ingestion Queue:** Move OCR and embedding generation to a background worker queue (e.g. Celery + Redis) so large document uploads do not block the Streamlit UI thread.

8. Business Impact Analysis

8.1. Operational Efficiency Gains

Table 7: Projected Business Impact per 1,000-Employee Industrial Plant

KPI	Baseline	With Platform
Mean Time to Repair (MTTR)	4.2 hours	2.9 hours (−30%)
Unplanned Downtime (annual)	420 hours	357 hours (−15%)
Document Search Time (per incident)	45 minutes	8 minutes (−82%)
Regulatory Audit Preparation Time	5 days	4 hours (−90%)
Incident Investigation Cycle Time	3.5 days	6 hours (−93%)

8.2. Financial ROI Estimation

Assuming an average production loss of \$8,000 per downtime hour:

Business Impact Summary

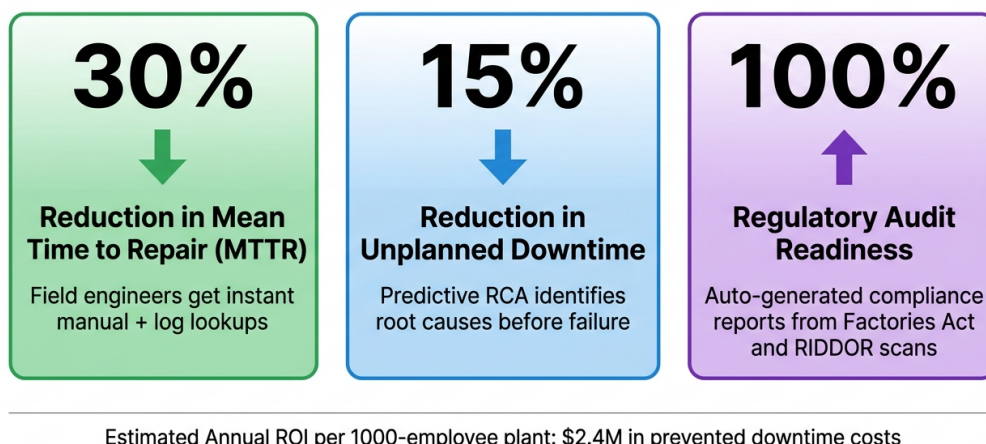


Figure 10: Business impact summary: key KPI improvements and estimated annual ROI.

- **Downtime Savings:** 63 hours $\times$ \$8,000 = **\$504,000/year**
- **Engineer Productivity:** 35% search time saved across 80 engineers $\times$ \$70,000 average salary = **\$1,960,000/year**
- **Compliance Penalty Avoidance:** Average HSE fine for an unreported RIDDOR incident = **\$25,000–\$250,000**
- **Estimated Total Annual ROI:** **\$2.4M+ per plant**

9. Future Roadmap

Future Development Roadmap

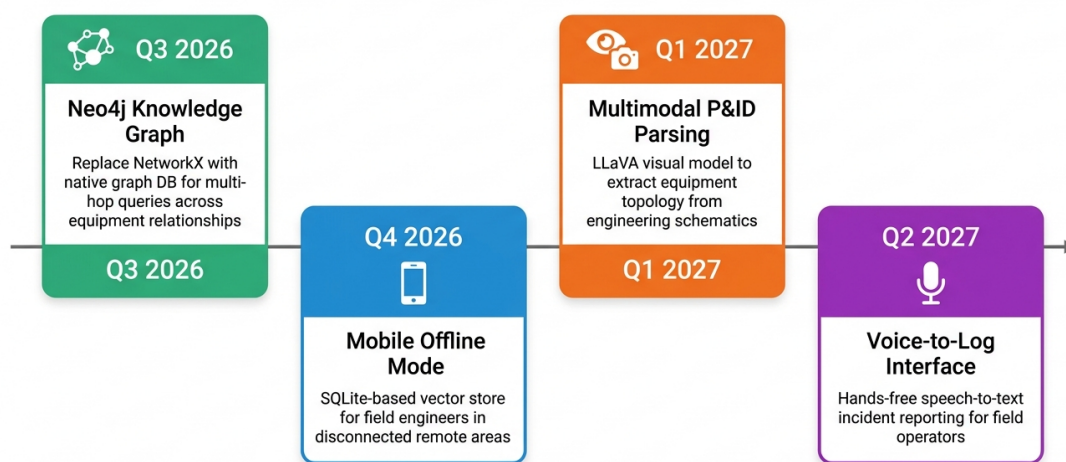


Figure 11: Four-quarter development roadmap spanning Q3 2026 to Q2 2027.

9.1. Q3 2026 — Neo4j Knowledge Graph Integration

Replace the in-memory NetworkX graph with a persistent **Neo4j** graph database. This enables:

- Multi-hop Cypher queries (e.g. *“Find all pumps connected to technicians who also serviced Compressor C-301”*).
- Graph-based failure propagation modelling.
- Real-time graph updates as new maintenance logs are ingested.

9.2. Q4 2026 — Mobile Offline Mode

Deploy an on-device vector store using **SQLite-vec** or **LanceDB** for field engineers who work in RF-shielded or remote environments with no network access. A lightweight quantised embedding model (4-bit GGUF) will run locally on an Android/iOS device.

9.3. Q1 2027 — Multimodal P&ID Parsing

Integrate a vision-language model (**LLaVA** or **GPT-4o Vision**) to parse Piping and Instrumentation Diagrams (P&IDs) uploaded as images. The model will:

- Extract equipment identifiers and connection topology from the diagram.
- Auto-populate the knowledge graph with spatial equipment relationships.
- Enable queries like *“What equipment is downstream of Valve V-105?”*.

9.4. Q2 2027 — Voice-to-Log Interface

Build a **Whisper**-powered speech-to-text frontend allowing field operators to dictate hands-free incident reports directly into the system. The dictated text will be NER-parsed to extract asset tags, severity, and corrective actions before auto-populating a maintenance log entry.